International Journal of Information Security (2026) 25:183
https://doi.org/10.1007/s10207-026-01350-x

REGULAR CONTRIBUTION

Planner-directed multi-agent LLMs with on-demand skill-equipped specialists for CTF-based autonomous penetration testing

Amir Amine[1] · **Samer Saab Jr.**[1]

Received: 12 August 2026 / Accepted: 12 October 2026
© The Author(s) 2026

Abstract

Large language model (LLM) agents increasingly automate offensive-security tasks, but success on isolated challenges does not establish reliable long-horizon autonomy. We study this distinction in authorized capture-the-flag (CTF) and wargame environments and introduce a planner-directed architecture in which a persistent non-executing planner delegates on demand to transient, context-isolated specialists with retrievable skills, persistent execution, compact memory, guardrails, and configurable model escalation. On all 200 challenges of the NYU CTF Bench test split, the system obtains 41 accepted flags (20.5%) and 29 leak-audited genuine solves (14.5%), against 4 (2.0%) and 3 (1.5%) for a stateful single-agent baseline. Both start from gpt-4o-mini, but only the planner-directed system could escalate to gpt-5-mini, which it did on 199 of 200 challenges and for 72.5% of spend. A controlled architecture-by-escalation experiment over a pre-registered subset of 80 challenges, crossing a resource-matched monolithic agent with the planner under matched budgets, was run in two independent formal passes. M0 solves 4/80 then 3/80, M1 22/80 then 19/80, P0 4/80 then 3/80, and P1 14/80 then 15/80. Pooling the two formal passes with challenge as the repeated unit, a clustered GEE shows a strong escalation effect (OR 8.79, 95% CI 3.39−22.81, $p = 8 \times 10^{-6}$), no architecture main effect (OR 1.17, $p = 0.34$), and a significant architecture-by-escalation interaction (OR 0.53, $p = 0.005$). Under the spend-constrained 340-action ceiling, the coordination cost of specialist delegation, context restart, and evidence folding offsets any additional solve-rate advantage from the planner-directed structure. On OverTheWire Bandit, the planner-directed system reaches 30/33 standalone and 24/33 sequentially, compared with 27/33 and 23/33 for the strongest single-agent mini backbone.

Keywords Autonomous CTF solving · LLM agents · Capture-the-flag · Multi-agent systems · Long-horizon autonomy · Benchmark integrity · Cybersecurity agents

1 Introduction

Penetration testing is fundamentally sequential. An operator must preserve state across reconnaissance, hypothesis formation, exploitation, credential recovery, pivoting, and verification, while adapting when an initially plausible line of attack fails. This makes autonomous penetration testing a demanding system-security-assurance task: capability is not only whether a model can suggest a correct command, but whether an agent can sustain a coherent closed loop over many actions and produce evidence that a security objective has actually been reached. Recent work in system security assurance identifies penetration testing as an important setting for adaptive AI-assisted evidence collection and security evaluation [17]. Generative AI has likewise been studied as an aid to human penetration testers, with benefits in explanation and workflow support but persistent concerns about correctness, bias, and uncontrolled behavior [7].

LLM-based penetration-testing systems have moved rapidly from assistance toward autonomous action. PentestGPT separates reasoning from generation and parsing to support penetration-testing workflows [5]; NYU CTF Bench provides a heterogeneous 200-challenge benchmark for offensive-security agents [12]; EnIGMA demonstrates that interactive security tools materially improve language-model agents [1]; and recent multi-agent systems use explicit planners, specialized executors, or knowledge retrieval to improve performance on CTFs and real vulnerabilities [13,

✉ Samer Saab Jr.
samer.saabjr@lau.edu.lb

[1] Department of Electrical and Computer Engineering, School of Engineering , Lebanese American University, Byblos, Lebanon

Published online: 05 November 2026

16, 19]. These systems establish that orchestration matters. They do not, however, remove two evaluation problems. First, isolated task success can substantially overstate what the same agent can accomplish in a state-carrying chain. Second, static public CTF benchmarks are vulnerable to both environmental shortcuts and pretraining or web contamination [10].

This paper studies both problems around a specific architectural question: how much persistent orchestration is needed to sustain long-horizon offensive-security work? We study this question through a planner-directed architecture and two complementary evaluation settings. The Deep Planner is a dynamic hierarchical multi-agent system with one persistent non-executing planner and no standing team. It delegates concrete tasks to fresh specialists created on demand, with exactly one specialist active at a time and no peer-to-peer specialist communication. Their expertise is supplied as category-scoped procedural skills loaded through progressive disclosure, while detailed trajectories are folded into compact evidence cards before control returns to the planner. OverTheWire Bandit is used to distinguish standalone task competence from sequential, state-carrying autonomy, while NYU CTF Bench evaluates the resulting system across a heterogeneous set of offensive-security tasks. Together, these settings examine both long-horizon continuity and broader end-to-end capability without requiring a permanently resident roster of agent contexts.

The NYU comparison is interpreted at the complete-system level. The planner-directed configuration combines task decomposition, specialist contexts, skills, memory mechanisms, guardrails, and adaptive escalation to larger models. That comparison therefore measures an integrated system and cannot by itself attribute the difference to orchestration. To address this directly we conduct a controlled architecture-by-escalation experiment (Sect. 4.3) in which a resource-matched monolithic agent and the planner-directed system are each run with escalation disabled and enabled under matched budgets, so that the experiment separates structure from escalation. We complement end-to-end outcomes with execution-behaviour analysis, provenance auditing of accepted flags, and provider billing records to characterize how the system operates and how its performance should be interpreted.

The paper makes three contributions:

1. *Architecture and system design.* We present a planner-directed hierarchical multi-agent architecture that retains a single persistent planning locus while instantiating skill-equipped specialists only on demand. The design combines transient context isolation, progressive skill disclosure, persistent execution, evidence folding, deterministic guardrails, checkpointed state, and adaptive model escalation.
2. *Audited full-system benchmark performance.* On the 200-challenge NYU CTF Bench test split, the planner-directed system obtains 29/200 leak-audited genuine solves (14.5%), compared with 3/200 (1.5%) for the stateful single-agent baseline. Both systems start from gpt-4o-mini, but only the planner-directed configuration has stall-triggered gpt-5-mini escalation. Across 3,926 specialist delegations, 92.5% are consistent with the supplied category hint.
3. *Integrity, long-horizon and cost analysis.* We report evaluation-validity and efficiency evidence alongside capability: a trace audit reduces 41 raw accepted flags to 29 genuine solves by excluding environment shortcuts, a demonstrated challenge-isolation protocol supports the remaining runs, and the planner-directed NYU run costs $136.83, or $4.72 per genuine solve. We evaluate long-horizon autonomy separately on OverTheWire Bandit, where the planner-directed system reaches 30/33 standalone and 24/33 sequentially.

Relation to the prior conference paper. This article extends our earlier study of stateful single-agent CTF solving [2]. Inherited from that work are the stateful single-agent controller, the OverTheWire Bandit evaluation protocol including the standalone-versus-sequential distinction, and the four single-agent Bandit rows of Table 6, which are reproduced unchanged. Everything else reported here is new: the planner-directed Bandit evaluation, and the planner-directed architecture and its skill layer, the entire NYU CTF Bench evaluation of both arms, the leak-audit protocol and the raw-versus-genuine metric, the controlled architecture-by-escalation experiment, the harness isolation and trace-retention protocol, the skill-corpus provenance audit, and the operational and cost analyses.

2 Related work

2.1 Automated penetration testing and security assurance

There exists an established line of work treating penetration testing as an autonomous decision problem rather than only a vulnerability-scanning task. Erdődi and Zennaro formalize web hacking as an agent-environment interaction in the Agent Web Model, using CTF-like abstractions to train reinforcement-learning agents [6]. Sommervoll et al. extend this direction to multiple archetypes of SQL-injection exploitation and show transfer among related vulnerability classes [15]. These studies are important predecessors because they make two choices that also govern our evaluation: they place an autonomous agent inside an executable environment, and they make success dependent on

a sequence of observations and actions rather than a static classifier output.

The recent literature broadens that view to AI-supported assurance. Hilario et al. study generative AI across penetration-testing activities and emphasize both operational benefits and risks [7]. Wen et al.'s systematic review positions adaptive AI-driven penetration testing within system security assurance, where active testing contributes evidence about the security posture of a system [17]. Our work sits at this intersection: it is not a new vulnerability detector, but an execution-grounded autonomous testing controller evaluated on authorized targets with explicit success verification and audit logs.

2.2 LLM agents for offensive security

PentestGPT demonstrated decomposing a penetration-testing workflow into reasoning, generation, and parsing modules can improve LLM effectiveness in interactive testing [5]. NYU CTF Bench subsequently made heterogeneous offensive-security evaluation more systematic by packaging 200 CSAW CTF challenges across cryptography, reverse engineering, binary exploitation, miscellaneous, web, and forensics [12]. OverTheWire Bandit has likewise been used to study autonomous shell-based CTF solving: Bakker and Hastings evaluate GPT-4o under isolated per-level execution [4], while Amine et al. introduce a stateful, checkpointed execution loop with a persistent shell and explicitly compare standalone with sequential, state-carrying evaluation on Bandit and Krypton [2]. Their results show that substantially more levels can be solved in isolation than when credentials and interaction state must be preserved across the full chain, establishing long-horizon state continuity as a distinct evaluation dimension rather than treating per-task success as equivalent to end-to-end autonomy.

Tooling has emerged as a major determinant of performance. EnIGMA equips agents with interactive tools such as debuggers and server-connection interfaces and shows that such tools substantially improve CTF solving [1]. A recent controlled study over NYU CTF further reports that the execution environment itself can shift performance materially, with a richer Kali-based tool environment outperforming Ubuntu in that study [11]. These findings are directly relevant to our limitations: our one-command, non-interactive execution contract improves determinism and auditability, but it also creates a hard ceiling on tasks that require sustained interactive I/O.

2.3 Planning, specialization, and knowledge in multi-agent systems

Several recent systems address long-horizon failure by distributing work across agents. HPTSA uses a planning agent that launches subagents to explore different vulnerability hypotheses and reports gains on real-world zero-day exploitation [19]. D-CIPHER combines a planner with heterogeneous executors and an auto-prompter, reporting 22.0% on NYU CTF Bench as part of a broader multi-benchmark evaluation [16]. CRAKEN instead emphasizes knowledge-based execution, retrieving CTF-derived knowledge and converting it into adaptive attack strategies [13].

The architecture in this paper differs in where persistence is placed. Rather than preserving a fixed set of role agents, it persists only the planner and the shared state graph. A task specialist is spawned into a fresh context, given the procedural skill relevant to that category, allowed to act, then discarded after returning a compact conclusion. Only one specialist is live at a time, and there is no peer-to-peer communication. The design therefore remains hierarchical and multi-agent in the ordinary systems sense, but seeks to reduce the amount of coordination state that must persist throughout the engagement.

This choice draws on two general agent ideas. ReAct interleaves language-model reasoning with environment actions [18], while Reflexion shows how compact verbal feedback can improve later attempts without retaining every low-level trace [14]. We operationalize the same broad principle through evidence cards and pull-style recall. For procedural knowledge, we use a progressive-disclosure skill pattern: a compact index is resident, while deeper notes are read only when the specialist decides they are relevant. The implementation follows the Agent Skills and Deep Agents software patterns [3, 9], but the scientific object evaluated here is the resulting security-testing system rather than those software abstractions themselves.

2.4 Evaluation validity and contamination

Public CTF benchmarks are reproducible but increasingly contamination-prone. CTFusion argues that reusing public challenges permits agents to encounter leaked solutions or exploit benchmark artifacts and proposes live CTFs as a streaming alternative [10]. This concern changes how a raw accepted flag should be interpreted. We therefore report both raw success and a stricter genuine success metric produced by post-hoc trace auditing. The audit detects environment shortcuts in our own runs; it cannot prove the absence of pretraining contamination. Accordingly, comparisons with prior published systems are contextual rather than treated as a controlled leaderboard.

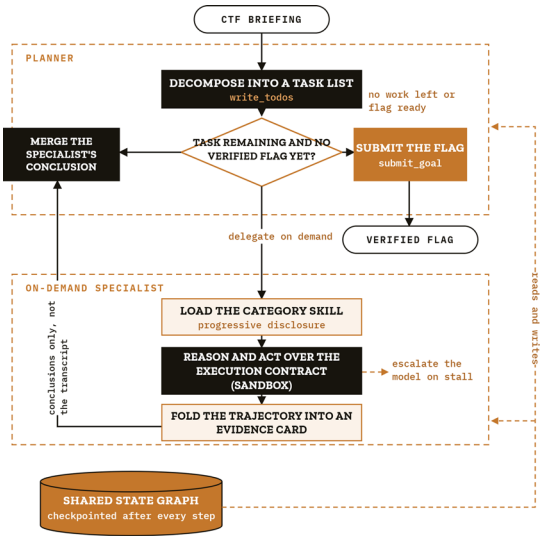


Fig. 1 Planner-directed Deep Agent architecture. A persistent non-executing planner delegates one task at a time to a fresh category specialist. The specialist receives category-scoped procedural skills, operates a shared persistent sandbox through structured tools, escalates on stall, and returns a compact evidence card. Exactly one specialist is active at a time

3 Planner-directed deep agent architecture

3.1 Design goals

The system is designed around four long-horizon constraints identified in prior work and reflected in our single-agent measurements: (i) planning must remain globally coherent across long trajectories; (ii) category-specific expertise should be available without placing all domain material in every prompt; (iii) verbose execution history must not grow without bound; and (iv) actions and submissions must remain auditable. A fifth practical goal is to run a small base model by default and to escalate to a stronger model when a specialist stops making progress. Section 5.5 shows that under the threshold used in the reported run the adaptive-escalation trigger fired on almost every challenge.

Figure 1 summarizes the resulting topology. One planner persists for the entire challenge. It does not execute shell commands. Instead, it maintains an explicit task list, chooses a category specialist for the next concrete task, receives the specialist's conclusion, and decides whether to re-plan, delegate again, or submit a candidate flag. Specialists are transient and isolated; only one exists at any given time.

3.2 Persistent planner and transient specialists

The planner is the sole locus of global reasoning. Its action space is deliberately narrow: maintain or revise the task list, delegate a task, and submit a final candidate. It is prevented from directly using the execution tools. This separation forces the planner to reason over progress at the level of hypotheses and tasks rather than terminal transcripts.

A delegation contains a task description and a category. The system can instantiate specialists for cryptography, web, binary exploitation, reverse engineering, forensics, miscellaneous, open-source intelligence, malware analysis, and AI/ML attack tasks. The specialist receives the delegated task plus a compact block of relevant prior evidence, but not the planner's complete history. It runs a local reason–act–observe loop until it succeeds, stalls, or exhausts its budget. At termination it returns findings and a suggested next step and is discarded. A subsequent delegation receives a new context, so an unproductive attack path does not automatically pollute the next attempt.

This lifecycle is intentionally different from a standing-agent committee. There is no planner-executor-summarizer conversation running continuously and no specialist-to-specialist messaging. Coordination is mediated by the planner and a structured state graph. This minimizes the number of long-lived contexts while still allowing specialization.

3.3 Procedural skills by progressive disclosure

Each specialist is paired with a category skill. A skill contains a compact technique index and references to deeper notes. Only the index is placed in the initial specialist context; the specialist can read a deeper note through a dedicated tool when a technique becomes relevant. Access is scoped so that a web specialist cannot arbitrarily read unrelated cryptography material.

The purpose is not to encode challenge answers. The skill layer supplies reusable procedures: reconnaissance patterns, common exploit-development steps, cryptanalytic checks, reverse-engineering workflows, and tool reminders. The distinction is important for both context efficiency and evaluation validity. The system's general strategy can be updated by editing the skill library without creating a permanently resident agent role.

The procedural content itself comes from CTF- SKILLS, a publicly available corpus of general CTF technique by Jagiełło [8], released under the MIT licence, comprising 118 notes across 11 packs. Our use of it at this layer is a delivery mechanism: one specialist per category, each equipped with the corresponding pack under progressive disclosure, with the planner delegating to the specialist matching the task. Section 4.6 reports the provenance and contamination audit of the corpus.

3.4 Execution contract

Every specialist uses the same three-tool execution contract:

1. *Execute.* Run exactly one non-interactive shell command inside a persistent pseudo-terminal session. A unique sentinel identifies true command completion and recovers the exit status. Commands that do not return within the bounded interval can be backgrounded; stuck PTY sessions can be reclaimed.
2. *Write.* Write a complete multi-line script to the shared workspace. Execution occurs in a separate subsequent action, preserving the one-action-per-turn rule.
3. *Submit.* Check a candidate flag against the benchmark's ground truth and return acceptance or a typed rejection reason.

The sandbox and challenge services share the environment required by the benchmark. The persistent terminal preserves working directory, artifacts, and remote-shell state between commands.

This contract favors determinism and auditing over interaction richness. It is well suited to scan–inspect–modify–execute workflows, but poorly suited to programs requiring ongoing bidirectional input such as interactive debuggers, SSL sessions, listener coordination, or exploit processes that must be driven repeatedly.

3.5 State, memory, and evidence folding

The state graph is the source of truth shared by planner and specialists. In addition to normal agent messages, it stores the current task, active specialist, latest command and output, accumulated evidence, candidate flag, verification state, and rejection reason. The graph is checkpointed after every node transition, making interrupted runs resumable and enabling post-hoc audits.

Memory is hierarchical. The live specialist history is bounded. Each command outcome is additionally stored as a compact abstracted case observation rather than a full raw transcript. When a specialist finishes, its trajectory is folded into an evidence card containing the task, principal observations, candidate artifacts, and recommended next action. A pull-style recall tool can retrieve relevant earlier observations. Thus, detailed outputs remain auditable in checkpoints while the active context receives a compressed representation.

3.6 Deterministic guardrails and adaptive escalation

Several guards constrain common failure loops. Duplicate commands or identical script bodies are blocked unless intervening evidence changes the context. Candidate submissions must match the expected flag shape and be grounded in recorded evidence. Repeated unsupported guessing is capped. A shared global step budget prevents unlimited delegation churn.

Both planner and specialists start on gpt-4o-mini. A specialist episode switches to gpt-5-mini when any of three conditions holds: the episode has emitted a placeholder submission; any submission has been rejected by the goal checker; or the episode has taken 12 tool actions without an accepted flag. Actions are counted per specialist episode over the three execution tools, and every rejected candidate counts, including those the submission guards block before goal checking. The switch is sticky: once triggered, the remainder of that specialist episode runs on the larger model, and a subsequent fresh specialist starts again on the base model. The planner never escalates and remains on gpt-4o-mini in every condition reported here.

4 Experimental method

4.1 Research questions

We organize the experimental suite around four questions:

RQ1: What end-to-end performance does the planner-directed, skill-equipped system achieve on a heterogeneous CTF benchmark relative to the stateful single-agent baseline?

RQ2: How does isolated task success differ from long-horizon sequential autonomy when credentials and execution state must be carried across dependent security tasks?

RQ3: What do behavioral traces, leak auditing, and billing records reveal about how the system operates and about the validity and cost of its observed success?

RQ4: Under matched skills, tools, guardrails, category hint and resource budgets, how much of the difference in RQ1 is attributable to the planner-directed structural package, and how much to the adaptive-escalation policy?

4.2 Ethical scope

All experiments are limited to authorized CTF environments designed for security training and benchmarking. The agents are not given arbitrary external targets, and success is defined by benchmark-specific goals. This containment is important because an autonomous offensive-security controller is dual use. We therefore report mechanisms at the level necessary for scientific reproduction and analysis while framing deployment beyond controlled environments as requiring explicit authorization, sandboxing, logging, and human governance.

4.3 Evaluation settings and experimental design

NYU CTF Bench. The main heterogeneous evaluation uses the 200-challenge NYU CTF Bench test split [12]: 52 cryptography, 51 reverse engineering, 39 binary exploitation (*pwn*), 24 miscellaneous, 19 web, and 15 forensics challenges. The stateful single-agent baseline and the DEEP PLANNER are evaluated on the same split.

The single-agent baseline uses GPT-4o-mini and no model escalation. The planner-directed system uses GPT-4o-mini as its planner and specialist base model and enables stall-triggered specialist escalation to GPT-5-mini. Both use persistent execution and explicit goal verification, but the planner-directed system additionally uses task decomposition, transient specialists, skills, context isolation, evidence folding, expanded guardrails, and adaptive escalation. These changes are bundled, and only the planner-directed arm could reach the larger model, so the controlled experiment below studies the bundle directly: architecture and adaptive escalation are varied separately under matched budgets, which shows what the planner-directed structure contributes and what escalation contributes.

NYU CTF Bench provides the heterogeneous end-to-end evaluation of the proposed system. We complement it with OverTheWire Bandit to isolate a different property of autonomy: whether progress can be sustained when credentials and execution state must be carried across a dependent sequence of tasks.

Controlled experiment. To separate architecture from model access we cross the two factors in a two-by-two design, giving four conditions: a resource-matched monolithic agent with escalation disabled (M0) and enabled (M1), and the planner-directed system with escalation disabled (P0) and enabled (P1). The escalation rule of Sect. 3.6 is applied in M1 and P1 with identical trigger conditions, threshold and stickiness, and the planner itself remains on the base model throughout. The monolithic control is not the single-agent baseline of the full-split comparison; it is built for this experiment and matched to the planner conditions on everything except organisation: the same frozen skill corpus scoped to the hinted category, the same execute, write and submit tools, the same sandbox, the same duplicate, evidence-grounding, flag-shape and repeated-guess guards, the same checkpoint backend, the same category hint and the same budgets, with an automated parity test asserting this. What it lacks is the planner-directed structural package: a persistent non-executing planner, delegation, transient specialist contexts, and specialist-to-planner evidence folding. Those four mechanisms vary together and are measured as one.

Repeating the full split under four conditions was beyond the available budget, so the experiment uses a pre-registered stratified subset of 80 challenges drawn proportionally to the split and selected before any result was seen: challenges are ordered within each category by a cryptographic digest of the submission identifier concatenated with the challenge identifier, and the category quota taken from the top of that order, which makes the draw deterministic, reproducible from the published manifest and blind to prior outcomes. Each challenge is attempted in two independent formal passes per condition from a fresh sandbox and state namespace. Between the two formal 2×2 passes, we additionally ran one standalone P1 robustness replication over the same 80 challenges under the same configuration and harness. The controlled revision study therefore comprises 720 challenge-runs: 320 in the first complete 2×2 pass, 80 in the standalone P1 robustness replication, and 320 in the second complete 2×2 pass. Independence between runs is enforced by the run name, since session identifiers derive from it and checkpoints are keyed by session. All conditions share an identical action budget, recursion cap, wall-clock timeout, retry policy, per-challenge spend and token ceiling, command timeout and output truncation, with ceilings calibrated on the development split alone and the terminating cap recorded for every run.

The primary endpoint is a trace-verified genuine solve, audited under the protocol of Sect. 4.4, with provenance-indeterminate outcomes counted conservatively as non-successes; secondary endpoints cover raw accepted flags, cost, calls and tokens, executed commands, escalation frequency, and duplicate and blocked-submission rates. Four contrasts and one interaction were fixed before any result was inspected: P0−M0 isolates structure without escalation, P1−P0 and M1−M0 isolate escalation within each architecture, P1−M1 isolates structure when both architectures escalate, and the architecture-by-escalation interaction tests whether the two substitute for one another. Online Resource 1 carries the pre-registration, the subset manifest and its composition.

Long-horizon Evaluation: OverTheWire Bandit. OverTheWire Bandit is a progressive Linux/SSH wargame in which the credential recovered at one level is required to access the next. We evaluate a stateful single-agent controller in two regimes. In standalone evaluation, each level is attempted independently with the required starting state supplied. In sequential evaluation, levels are attempted in order and the agent must preserve credentials, shell state, and progress across the chain. The same stateful controller is used in both regimes; therefore the difference isolates the practical cost of chaining tasks rather than comparing a stateful system against a stateless implementation. The four single-agent backbone runs are reproduced unchanged from Amine et al. [2]; the planner-directed run was executed for this article. All of them use the same public wargame, the same level set, the same tool set and the same persistent-session execution model, which is the equivalence relevant to the

comparison. Because the wargame rotates level passwords and is externally hosted, the inherited and new runs did not use identical credentials and level content cannot be certified identical across execution windows.

The evaluated single-agent backbones are GPT-4o-mini, GPT−4.1-mini, GPT-5-mini, and GPT-4o. We additionally cite the published standalone GPT-4o result of Bakker and Hastings [4] as an external reference.

4.4 Outcome definitions and leak audit

A raw solve is any challenge for which the goal checker accepted the submitted flag. A genuine solve requires that the recorded trajectory recovered the flag by solving the intended challenge rather than through a benchmark-environment shortcut. For every raw solve, the audit inspected the model/tool trace and classified the provenance of the accepted flag.

The audit applies four environment-leak checks: reading a dataset mount containing solution material, reading state from a live challenge container, reading an orchestrator or provisioning script, and obtaining a flag baked into an accessible image or artifact. A fifth outcome, apparent memorisation from pretraining, is recorded separately, since it is not an environment leak. The primary performance metric in this paper is genuine solve rate, with raw rate reported to show the magnitude of the validity correction.

Both arms run under a trace store that records, for every challenge attempted, every generation input, every tool-call input, and the captured output of every executed shell command. Each accepted flag is classified by where its value first appears in that record, ordered by time, so a flag read from an environment surface is separated from one the agent derived. No classification is indeterminate. Online Resource 1 gives the per-flag classification and the coverage evidence.

The whole protocol was developed while iterating on the NYU CTF development split and applied unchanged to the test split, before any test-split trace was inspected. Online Resource 1 documents the development-split iteration record.

This audit does not remove the separate risk of pretraining contamination from public challenge write-ups. CTFusion's results motivate treating public-benchmark scores cautiously [10]; a fresh or live challenge suite would be needed to address that threat directly.

4.5 Harness isolation

Challenge isolation is a property the harness must provide, and the audit found one case where it did not: scratch storage shared by the sandbox persisted across challenges and escaped the per-challenge reset, so a flag written during one challenge could be read during a later one. The channel is filesystem state, not planner memory, evidence cards or checkpoints, which distinguishes a provisioning defect from a failure of the agent's own state boundaries.

Two properties close it. Shared scratch storage is cleared and surviving processes terminated at every challenge materialisation, and the four checked surfaces of Sect. 4.4 are unreachable from the container hosting the agent's shell. Both are asserted by test: a canary suite plants marker state on ten surfaces and requires the next challenge to observe none of it, with the reset disabled as a negative control, while a per-shard denial test confirms each checked surface is closed and a positive control confirms the workspace remains usable.

During the controlled-run audit, we identified a second infrastructure defect introduced by the repair: the shared scratch volume also exposed the container runtime content store through the host `/tmp` mount. One P1 attempt in the first formal 2×2 pass was therefore declared void and was not credited as a genuine solve; the pass-level result remains reported over the fixed 80-challenge subset as 14/80. After the volume layout was corrected and the denial tests passed, the same challenge was rerun under the corrected harness in both later P1 executions. It was not solved in the standalone 13/80 robustness replication and was solved genuinely in the second formal 2×2 pass, whose P1 total was 15/80.

Every challenge whose accepted flag had crossed an environment boundary was re-run under the repaired configuration. Across 17 isolation re-runs, the bleed did not recur, no run reached a denied surface, three challenges solved again genuinely and 14 did not. Online Resource 1 reports the canary and denial results, the re-run outcomes and the resulting reclassifications.

4.6 Skill-corpus provenance

The skill corpus is CTF- SKILLS [8], referenced under the MIT licence, and used as published. The corpus is third-party, adopted unmodified at a single public commit that predates the first test run of this evaluation by three months, so the version we ran can be fetched and compared directly. Per-note digests are listed in Online Resource 1.

The system built around the corpus followed a development-split-only design policy. Prompts, guardrails, routing rules and the specialist configuration were developed against the NYU CTF development split across five recorded iteration cycles. Test-split traces were read after the reported run, for the leak audit, under a protocol fixed beforehand on the development split. The corpus was used as published throughout. Because the corpus derives from public CTF write-ups, we scanned it for benchmark material and report the outcome with the results, in Sect. 5.3.

4.7 Model, software and pricing provenance

Execution windows for the original full-split NYU arms and the planner-directed Bandit evaluation are recorded in Online Resource 1. The original full-split run was executed through OpenRouter, whose archived configuration names the aliases `gpt-4o-mini` and `gpt-5-mini`; OpenRouter returned the alias rather than the dated identifier on most calls, so a resolved dated snapshot is not available for every original generation. The revision experiments use the OpenAI API directly and record the dated snapshots `gpt-4o-mini-2024-07-18` and `gpt-5-mini-2025-08-07` on every call. Across the stored traces, exactly one dated snapshot per model family appears; the original-run store also contains `openai/gpt-5-mini-2025-08-07` returned by OpenRouter during the alias-based execution window. We retain the residual caveat that an alias can in principle be repointed. Costs are provider-billed amounts as recorded; Online Resource 1 reports the provider rate schedule and the date used for cost validation.

The code that produced the reported results is fixed by three public tagged repositories. The evaluation harness, analysis, and released artefacts are at https://github.com/iamamiramine/gencyber-Evaluation, release `ijis-revision-v1.0`, commit `cdd5d97f0aa369e29c6ee9280f6e9eaaf3a63d11`; the agent is at https://github.com/iamamiramine/gencyber-Agent, release `gencyber-agent-v1.0`, commit `761b1fb7465d107736be783c126fe713a4a732aa`; and the sandbox and challenge API are at https://github.com/iamamiramine/gencyber-Workbench, release `gencyber-workbench-v1.0`, commit `5503caafa2afbeed0f1822cc7416e024402f58de`. Together, the release tags and exact commit identifiers fix the source revisions used for the reported results. Online Resource 1 records the corresponding environment-pinning information.

4.8 Execution behaviour and cost

End-to-end outcome statistics use all 200 NYU CTF Bench challenges. Every run reported in this article continues until one of the stated caps ends it. This holds in both full-split arms and in every condition of the controlled experiment. The behavioral aggregates are computed from the trace store, which retains every generation input, every tool-call input, and the captured output of every executed shell command for all 200 challenges. Unless stated otherwise, behavioral quantities such as executed commands, specialist delegations, hint-consistent routing, escalations per episode, accepted-versus-blocked submissions, and duplicate-command rate are reported over all 200 challenges.

Billing records provide per-category monetary cost for all 200 planner-directed challenges and all 200 single-agent challenges. The recorded planner-directed provider bill totals $136.83 across these 200 challenge-level billing records. We report provider-billed totals exactly as recorded and derive cost per raw and genuine solve arithmetically. The planner-directed run produced 87,372 model calls. About 93.2% of its input-token volume was billed at the cache-read rate because the system repeatedly reused a stable prefix; this is reported as an implementation property, not as a general cost guarantee for other providers or caching policies.

Additional per-challenge outcomes, archived configuration files and prompts, complete level-wise Bandit outcomes, and audit details are provided in the Electronic Supplementary Material (Online Resource 1).

Online Resource 1. Electronic supplementary material containing the full per-challenge outcome record, archived experiment configuration and prompts, leak-audit details, billing/behavioral summaries, and complete level-wise Bandit outcomes.

Table 1 Planner-directed NYU CTF results by category

Category	*N*	Raw	Genuine	Genuine %
Cryptography	52	10	6	11.5
Reverse eng	51	11	9	17.6
Pwn	39	8	5	12.8
Miscellaneous	24	5	4	16.7
Web	19	4	2	10.5
Forensics	15	3	3	20.0
Total	**200**	**41**	**29**	**14.5**

Bold values indicate totals or aggregate results

5 Results

The results follow the four research questions. We report end-to-end performance on NYU CTF Bench with paired statistics (RQ1), the controlled architecture-by-escalation experiment (RQ4), escalation exposure and resource decomposition, the standalone-versus-sequential distinction on Bandit (RQ2), operational behaviour and cost (RQ3), and finally the contextual placement against previously reported systems.

5.1 RQ1: end-to-end NYU CTF performance

Table 1 and Fig. 2 report planner-directed performance by category. The system submits 41 accepted flags (20.5%) before auditing and retains 29 as genuine (14.5%). Genuine rates range from 12.8% on binary exploitation to 20.0% on forensics. Reverse engineering contributes the largest absolute number of genuine solves (9), followed by cryptography (6).

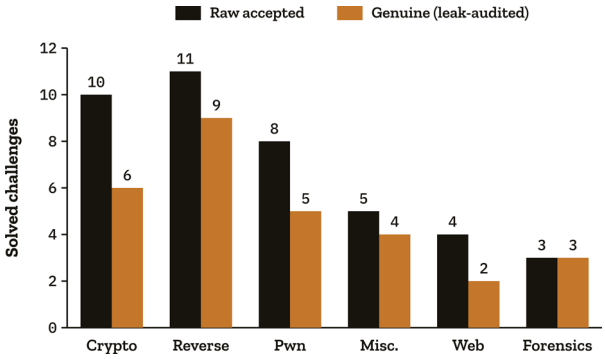


Fig. 2 Raw accepted and leak-audited genuine solves by NYU CTF category. The difference is excluded from the primary capability metric because the accepted flag was obtained through environment shortcuts rather than the intended challenge path

Table 2 Direct comparison on the same 200 NYU CTF test challenges. Both arms begin on GPT-4o-mini; only the planner-directed arm could escalate to GPT-5-mini

System	Raw	Genuine	Genuine rate
Stateful single agent	4	3	1.5%
Planner + specialists	41	29	14.5%

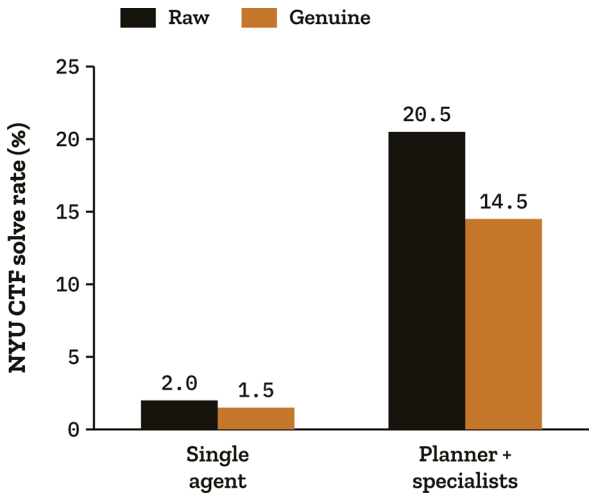


Fig. 3 Complete-system comparison on the same NYU CTF test split. Both arms start from the same GPT-4o-mini base model, but the planner-directed arm additionally has stall-triggered GPT-5-mini escalation and the architectural mechanisms of Sect. 3. The controlled experiment of Fig. 4 separates these two factors

The same-test-set complete-system comparison is shown in Table 2 and Fig. 3. The stateful single agent obtains 3 genuine solves (1.5%) and 4 raw solves (2.0%). The planner-directed system obtains 29 genuine solves (14.5%) and 41 raw solves (20.5%). The 3 genuine single-agent solves occur in reverse engineering, forensics, and miscellaneous, one each.

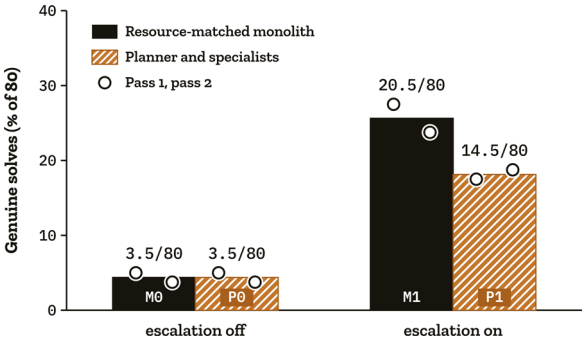


Fig. 4 Controlled architecture-by-escalation experiment over the pre-registered 80-challenge subset. Bars show the mean genuine solve rate across the two formal passes for each condition, with the individual pass rates overlaid as points. Challenge-clustered bootstrap 95% intervals are reported in Online Resource 1. The additional standalone P1 robustness replication is reported in the text

5.2 Paired comparison of the two arms

Both arms were evaluated on the same 200 challenges, so their outcomes are paired and are reported as such. The two systems agree on 172 challenges, 2 solved genuinely by both and 170 by neither, and disagree on 28: 27 of the planner-directed system's 29 genuine solves are challenges the single agent fails, and 1 of the single agent's 3 is a challenge the planner-directed system fails. An exact McNemar test on those 28 discordant pairs gives $p = 2.16 \times 10^{-7}$. The overlap is the more informative quantity: 2 of the 3 genuine single-agent solves are also solved genuinely by the planner-directed system, so the planner-directed system extends coverage instead of succeeding on a different subset. Online Resource 1 names all three.

Under the reported configurations the planner-directed system is materially stronger at the complete-system level. That comparison bundles task decomposition, specialists, skills, context isolation, guardrails, memory mechanisms and adaptive escalation, including access to a larger model that the baseline never had.

5.3 Skill-corpus exposure of the genuine solves

The corpus derives from public CTF write-ups, and the scan reflects that. Searching for benchmark challenge identifiers and titles, flag wrappers and literal flags, and distinctive test-split filenames, hosts, ports, keys and constants, CSAW appears in 31 of the 118 notes, and three test-split challenges are named with technique attached.

What bounds the consequence is per-solve exposure. A specialist meets the corpus at two layers: its system prompt is a pack index, so every episode sees one, and deeper notes are opened on demand. For every genuine solve we recorded the pack delegated to and the notes opened, and resolved

Table 3 Controlled architecture-by-escalation results on the pre-registered stratified subset. M = resource-matched monolith, P = planner and specialists; 0 = escalation off, 1 = escalation on. Counts are trace-verified genuine solves out of 80 challenges in two independent passes. The model pools both passes over 640 observations in 80 clusters; challenge-clustered bootstrap intervals on the risk differences are in Online Resource 1. An additional standalone P1 robustness replication, conducted between the two formal 2×2 passes, yielded 13/80 genuine solves. Across the three P1 executions (14/80, 13/80, and 15/80), the mean was 14.0 ± 1.0 solves. The complete controlled study comprised 720 challenge-runs

Genuine solves out of 80				*GEE logistic, clustered by challenge*			
Condition	Pass 1	Pass 2	Mean	Term	OR	95% CI	p
M0, monolith, esc. off	4	3	3.5	Escalation	8.79	3.39−22.81	8×10^{-6}
M1, monolith, esc. on	22	19	20.5	Architecture	1.17	0.85−1.62	0.34
P0, planner, esc. off	4	3	3.5	Arch. × escalation	0.53	0.34−0.82	0.005
P1, planner, esc. on	14	15	14.5	Pass	0.83	0.63−1.10	0.19

every CSAW reference in that material against the test split. All 29 genuine solves were exposed at the first layer and 14 at the second, with 15 opening no notes. Across both layers, 1 item in play cites a test-split challenge, and it names a challenge other than the solve that opened it, so it cannot have supplied that answer. None of those three challenges is a genuine solve: two were failures, and the third is among the leaked solves, excluded on independent grounds because the transcript records the agent reading the flag from the dataset mount. A broader search across the corpus for the name of any genuine solve, independent of CSAW context, returned 5 matches, each an ordinary-word collision confirmed individually. Online Resource 1 names the three challenges and gives the per-pack manifest.

5.4 RQ4: controlled architecture-by-escalation experiment

Table 3 and Fig. 4 report the two-by-two experiment over the pre-registered subset of 80 challenges, run in two independent formal passes. With escalation disabled the two architectures solve the same number of challenges, 4 each in the first pass and 3 each in the second. With escalation enabled the monolith solves 22 and 19 against the planner's 14 and 15. Pooling the two formal passes with challenge as the repeated unit, a GEE logistic model gives an escalation odds ratio of 8.79 (95% CI 3.39−22.81, $p = 8 \times 10^{-6}$), an architecture odds ratio of 1.17 (0.85−1.62, $p = 0.34$), an architecture-by-escalation interaction of 0.53 (0.34−0.82, $p = 0.005$) and no pass effect (0.83, $p = 0.19$). Challenge-clustered bootstrap intervals on the risk differences, reported in Online Resource 1, exclude zero for both escalation contrasts and for the structural contrast with escalation enabled, and include zero for the structural contrast with escalation disabled. All four contrasts keep their direction across passes. Between the two formal 2×2 passes, an additional standalone P1 robustness replication over the same 80 challenges yielded 13/80. Thus, in chronological order, the three P1 executions yielded 14/80, 13/80, and 15/80, with a mean of 14.0 and a standard deviation of one challenge. Under matched tools, corpus, guards, hint, and budgets on the controlled subset, adaptive escalation produces a strong positive solve-rate effect, while the planner-directed structural package does not provide an additional solve-rate advantage at this action budget.

Table 4 Adaptive-escalation exposure in the full-split planner-directed run

Exposure level	Value
Challenges with ≥ 1 larger-model call	199/200
Escalated specialist episodes	91.9%
Larger-model calls	56,312 of 87,372
Larger-model share of calls	64.5%
Larger-model share of spend	72.5% ($99.25)

5.5 Escalation exposure and resource decomposition

Table 4 quantifies how often the adaptive-escalation rule fired in the full-split planner-directed run, at four levels of granularity. At least one trigger condition was met on 199 of 200 challenges, the larger model accounted for 64.5% of model calls and 72.5% of spend, and 91.9% of specialist episodes escalated. Among the 29 genuine full-split solves, the first decisive genuine-solve evidence appeared after escalation in 22 cases, before escalation in 6, and on the sole challenge with no escalation in 1. This timing analysis is descriptive rather than causal because each full-split challenge was run under only one configuration.

In the first paired pass, P1 solved 11 challenges that P0 did not. Across the two formal 2×2 passes, there were 24 P1-solved/P0-failed challenge-pass occurrences; collapsing repeated occurrences of the same challenge gives 17 distinct challenges exhibiting this pattern.

Table 5 decomposes calls and cost by role and model, which answers how the run's scale divides between the

Table 5 Model calls and provider-billed cost by role and model for the planner-directed full-split run. Unattributed calls are those whose parent span was lost at ingest, so their role cannot be resolved from the trace; they are reported separately rather than assigned, which keeps the planner's zero escalations exact

Role / model	Calls	Cost ($)	% cost
Planner, base model	6,397	13.82	10.1
Specialist, base model	23,763	21.74	15.9
Specialist, escalated	56,312	99.25	72.5
Unattributed	900	2.02	1.5
Total	**87,372**	**136.83**	**100**

Bold values indicate totals or aggregate results

Table 6 OverTheWire Bandit completion totals. Single-agent runs are inherited unchanged from the stateful evaluation of Amine et al. [2]; the external GPT-4o result is from Bakker and Hastings [4]; the planner-directed run was newly executed for this article. Gap = standalone − sequential

Configuration	Seq	Stand	Gap	Provenance
GPT-4o-mini	12/33	19/33	7	inherited
GPT−4.1-mini	16/33	22/33	6	inherited
GPT-5-mini	23/33	27/33	4	inherited
GPT-4o	16/33	20/33	4	inherited
GPT-4o ext. [4]	–	20/33	–	published
Deep Planner	24/33	30/33	6	new

planner, the base-model specialists and the escalated specialists. The planner is a small share of total spend despite being the only persistent context, because it neither executes commands nor consumes command output; the escalated specialist calls dominate.

5.6 RQ2: long-horizon autonomy on OverTheWire Bandit

Bandit complements the heterogeneous NYU CTF evaluation by isolating state continuity across a dependent sequence of Linux and SSH tasks. Table 6 reports both standalone evaluation, in which each level is attempted independently with its required starting state supplied, and sequential evaluation, in which credentials and shell state must be carried forward through the chain.

Every stateful single-agent configuration solves more levels under standalone evaluation than under sequential chaining. The largest gap occurs for GPT-4o-mini, which falls from 19/33 to 12/33, while the strongest single agent, GPT-5-mini, falls from 27/33 to 23/33. The planner-directed configuration exhibits the same distinction, reaching 30/33 standalone and 24/33 sequentially. These results reinforce that successful local actions do not by themselves establish reliable end-to-end autonomy when earlier credentials, shell state, and interaction outcomes determine later progress.

The planner-directed configuration also extends the Bandit totals relative to the strongest single agent, from 27/33 to 30/33 standalone and from 23/33 to 24/33 sequentially. Because the single-agent runs are inherited from the earlier study, the provenance column applies to this difference: the runs share the wargame, level set and starting-state convention, but were executed at different times, and a one-level sequential difference is within the range that ordinary run-to-run variation could produce. The smaller separation on Bandit than on NYU CTF is consistent with the different demands of the two settings. Bandit is a comparatively homogeneous Linux progression in which the attack category is usually evident, whereas NYU CTF repeatedly changes task type, artifacts, tools, and exploitation strategy. Bandit therefore primarily stresses state continuity, while NYU CTF places greater demand on task decomposition and specialist selection.

The level-wise outcomes also expose an execution-surface limitation shared with the NYU experiments. Sequential progress reaches level 23 before failing at level 24, and several later failures require interactive behaviors that are poorly matched to the one-non-interactive-command-per-turn contract. Standalone evaluation recovers levels 26–31 once the required starting state is supplied independently. The full level-wise record is provided in Online Resource 1.

5.7 RQ3: operational behavior and cost

The end-to-end results establish what the system solves; the trace and billing records characterize how that performance is produced. We therefore examine specialist routing, escalation, duplicate and submission behavior, and provider-billed cost under the same planner-directed evaluation.

5.7.1 Hint-consistent routing and execution behavior

The behavioral record provides evidence that the architecture is being used as intended. Across these challenges, specialists execute 110,750 commands and receive 3,926 delegations. Overall, 92.5% of recorded delegations are consistent with the category hint. The benchmark category is supplied to the planner directly in a `Challenge category (routing hint): X` field, with an instruction to delegate to the corresponding specialist. The rate therefore measures whether the implementation follows its own routing instruction, and is a sanity check that specialists were genuinely exercised. Table 7 gives category-level measurements and Fig. 5 plots the rates.

Hint-consistent routing is strongest for miscellaneous (99.0%) and cryptography (96.3%), and weakest for forensics (79.3%) and web (86.1%). The first delegation is

Table 7 Execution behaviour by category over all 200 planner-directed challenges. Three denominators are in play and are not interchangeable: delegations are task-tool calls, episodes are subagent invocations that actually started, and submissions are candidate flags offered to the guards. Hint-consistent routing and escalated-episode share are over episodes; the oracle ratio is over submissions; duplicate rates are over commands. "Blocked" is the duplicate guard refusing a command; "repeat" is the agent re-issuing text it had already issued, whether or not the guard fired

Category	*N*	Commands	Deleg	Episodes	Hint-consist. %	Escal ep. %	Oracle / attempt	Dup blocked %	Dup. repeat %
Cryptography	52	23,304	1,005	1,005	96.3	93.9	97/577	0.3	57.9
Reverse eng	51	38,966	1,142	1,067	94.1	91.2	127/749	0.8	59.1
Pwn	39	25,093	730	675	89.0	90.2	72/376	0.4	58.4
Miscellaneous	24	9,897	391	391	99.0	91.6	32/214	0.3	54.7
Web	19	7,582	397	373	86.1	92.5	34/243	0.7	59.1
Forensics	15	5,908	261	261	79.3	91.2	28/159	0.2	49.5
Total	**200**	**110,750**	**3,926**	**3,772**	**92.5**	**91.9**	**390/2,318**	**0.5**	**57.8**
Single agent	200	49,170	0	—	—	—	126/311	0.0	92.8

Bold values indicate totals or aggregate results

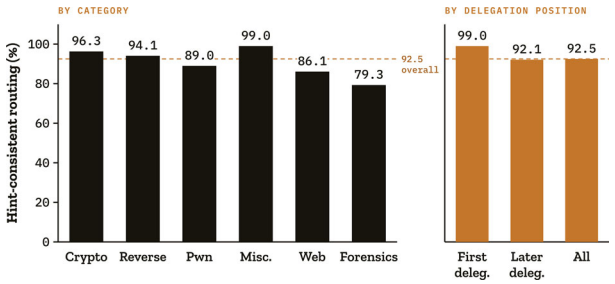


Fig. 5 Hint-consistent specialist routing by delegation position. The planner is supplied the benchmark category, so these rates measure compliance with the supplied hint rather than independent specialty inference. Compliance is near-perfect on first contact and decays across later delegations, which occur after a specialist has stalled

Table 8 Planner-directed run cost by category

Category	Cost ($)	Records	$/record
Reverse	41.94	51	0.82
Crypto	34.64	52	0.67
Pwn	27.16	39	0.70
Misc	15.27	24	0.64
Web	9.15	19	0.48
Forensics	8.67	15	0.58
Total	**136.83**	**200**	**0.68**

Bold values indicate totals or aggregate results

hint-consistent 99.0% of the time across 200 first delegations, while the 3,572 later delegations fall to 92.1%. The first delegation simply follows the supplied hint, whereas later ones occur after a specialist has stalled and the planner begins searching other categories. That drift is where the planner departs from its instruction, and it is also where opportunistic exploration, and therefore cross-boundary read risk, concentrates.

Adaptive escalation is how most episodes run: 91.9% of specialist episodes escalate, and an escalated episode issues 14.93 larger-model calls on average.

The submission guards are the dominant filter on candidate flags: 83.2% of the 2,318 attempted submissions are blocked before reaching the oracle, leaving 390 checked. The single-agent arm blocks 59.5% of 311. This is desirable from an auditability perspective, though the leak audit also found a case where guard interactions suppressed a genuinely solved challenge, illustrating that deterministic rules can introduce false negatives.

The duplicate guard blocks 0.5% of commands. Separately, 57.8% of commands repeat text the agent had already issued, whether or not the guard fired. The contrast with the baseline is informative: the single-agent arm has no duplicate guard, so it blocks 0.0% while 92.8% of its commands are repeats. The guard is doing work for which the baseline has no equivalent.

The same traces also reveal the computational price of this behavior. In particular, extensive delegation and frequent escalation increase per-challenge expenditure relative to the single-agent baseline, motivating an outcome-normalized cost comparison.

5.7.2 Cost and cost effectiveness

Table 8 reports recorded provider-billed cost. The planner-directed run totals $136.83 across 200 challenge-level billing records, or approximately $0.68 per recorded challenge bill. Dividing by the audited outcomes gives $3.34 per raw solve and $4.72 per genuine solve. The single-agent run costs $70.48 across all 200 challenges, or $0.352 per challenge and $23.49 per genuine solve.

The planner-directed system is therefore about 1.9 times more expensive per challenge record than the single agent,

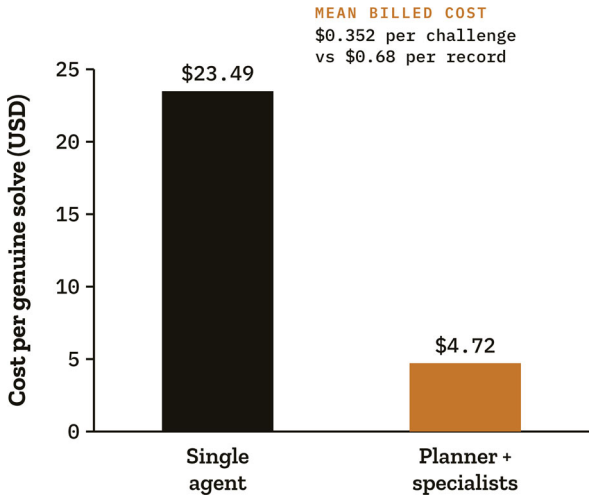


Fig. 6 Cost per leak-audited genuine solve on the same NYU CTF test split. The planner-directed system costs more per attempted challenge but less per genuine solve because it succeeds more often

but about 5.0 times cheaper per genuine solve ($4.72 versus $23.49). Figure 6 visualizes this reversal. Cost per solve is an outcome-dependent metric and should not be generalized beyond this benchmark, but it is useful here because the cheaper single-agent trajectory often spends its full budget without producing a verified result. The larger model accounts for 72.5% of spend, so the cost-per-genuine-solve advantage reflects the adaptive-escalation policy as well as the architecture, and the P1−P0 contrast in Table 3 separates the two.

Approximately 93.2% of the planner-directed run's input-token volume was billed at the provider's cache-read rate (about 1.58 billion cached input tokens). The architecture repeatedly reuses a stable system / skill / evidence prefix, so caching is an important implementation-level reason that the context-heavy run remains within the reported budget. We do not estimate a universal counterfactual uncached cost because provider prices and cache behavior can change.

Taken together, the RQ3 measurements show an architecture that is actively exercising its intended control mechanisms rather than merely exposing them in configuration: delegation is frequent and predominantly hint-consistent, escalation is routinely invoked on stalled episodes, guardrails materially filter candidate actions, and the added computation yields a lower observed cost per genuine solve despite a higher cost per attempted challenge.

5.8 Contextual placement relative to prior NYU CTF systems

Table 9 places the genuine 14.5% result alongside previously reported raw outcomes for prominent NYU CTF systems. These values are not directly controlled comparisons: model backbones, tools, attempt budgets, environments, knowledge access, and contamination controls differ. We include them only to show the performance regime in which the present system operates.

A useful interpretation is not that a mini-tier system "matches" any particular frontier model under equal conditions; the conditions are not equal. Rather, the result shows that the planner-directed design reaches a non-trivial performance band on a heterogeneous benchmark while using a small base model and adaptive escalation. Recent evidence that environment/tool selection can move NYU CTF performance by several percentage points [11] reinforces the need for this cautious interpretation.

6 Discussion

6.1 What the available evidence supports

The full-split result establishes that the integrated planner-directed system solves substantially more NYU CTF challenges than the stateful single-agent baseline: 14.5% against 1.5% genuine, an absolute difference of +13.0 percentage points. That comparison is at the complete-system level: it bundles planner structure, skills, memory, guardrails and access to a larger model, and the two arms ran under different harness configurations. The controlled run of Sect. 4.3 was designed to distinguish the matched effects of structure and adaptive escalation.

In the controlled experiment, under matched skills, tools, guards, category hint and budgets, the two architectures remain level without escalation across both formal passes (4 versus 4 and 3 versus 3), while escalation increases solve counts in both architectures (M1: 22 and 19 versus M0: 4 and 3; P1: 14 and 15 versus P0: 4 and 3). The matched analysis therefore shows a strong positive contribution from adaptive escalation, while the planner-directed structural package does not provide an additional solve-rate advantage at this action budget.

The architecture also behaves as designed: the planner performs thousands of delegations that predominantly follow the supplied category hint, specialists commonly escalate after stalls, the duplicate and evidence guards actively filter behavior, and checkpointed traces support a post-hoc leak audit. Together, these observations show that the design is a functioning long-horizon security-testing system rather than a prompt-only wrapper around a single model call.

Viewed together, the two evaluation settings separate heterogeneous task capability from state-carrying continuity. NYU CTF Bench stresses heterogeneous end-to-end problem solving and yields a large complete-system separation. Bandit instead stresses state continuity within a comparatively homogeneous sequence. There, both single-agent and

Table 9 Contextual comparison with previously reported NYU CTF results. Prior values are published/raw success rates; our primary value is leak-audited genuine success. Protocol and model differences preclude leaderboard-style conclusions

System / model	Crypto	Reverse	Pwn	Web	Forensics	Misc	Overall
NYU CTF baseline / GPT-4 [12]	0.0	9.8	7.7	5.3	6.7	0.0	≈5.0
EnIGMA / GPT-4-Turbo [1]	1.9	9.8	5.3	0.0	13.3	16.7	7.0
EnIGMA / Claude−3.5 [1]	7.5	17.6	18.4	0.0	20.0	16.7	13.5
D-CIPHER / GPT-4o [16]	5.8	13.7	7.7	10.5	13.3	16.7	10.5
D-CIPHER / Claude−3.5 [16]	15.4	29.4	12.8	5.3	20.0	25.0	19.0
CRAKEN / Claude−3.5 [13]	15.4	27.5	20.5	15.8	26.7	29.2	22.0
DEEP PLANNER / GPT-4o-mini to GPT-5-mini (genuine)	11.5	17.6	12.8	10.5	20.0	16.7	**14.5**
DEEP PLANNER / GPT-4o-mini to GPT-5-mini (raw)	19.2	21.6	20.5	21.1	20.0	20.8	20.5

Bold values indicate the primary leak-audited genuine solve rate of the present study

planner-directed configurations perform better when levels are isolated than when they must be chained, and the planner-directed configuration reaches 24/33 sequentially compared with 23/33 for the strongest single agent. Taken together, the results suggest that the utility of orchestration depends on the demands of the task: specialization and task decomposition have more opportunity to contribute when attack categories, artifacts, and techniques vary, while persistent state remains important even when the task family is comparatively uniform.

6.2 How often adaptive escalation fires

Under the threshold actually used, at least one trigger condition was met on 199 of 200 challenges, and the larger model accounted for 64.5% of calls and 72.5% of spend. Adaptive escalation therefore operates across the run: the policy runs the cheap model until a cheap signal of non-progress appears, then commits the remainder of the episode to the expensive one.

6.3 Coordination cost under a fixed action budget

The controlled result is consistent with a coordination-overhead explanation under a spend-constrained action budget. The common 340-action ceiling was chosen to bound per-challenge expenditure under a matched resource envelope, rather than because 340 actions was assumed to be sufficient for either architecture. Delegating a task, restarting a specialist in a fresh context, and folding its trajectory back into an evidence card all consume actions before directly advancing the challenge, so these coordination operations draw from the same allowance that the monolithic agent can spend directly on challenge execution. This constraint affected the planner-directed condition more often: P1 reached the action ceiling on 60.0% of challenges, compared with 46.9% for M1, and its median run used 301 tool actions versus 240 for M1. The traces therefore support interpreting the controlled comparison as performance at a deliberately cost-constrained operating point rather than as an unconstrained comparison of the two architectures.

The repeats support reading the contrasts at the level of the rate: no cell shows a material pass-level shift, every contrast keeps its direction, and the model finds no pass effect, while per-challenge agreement within a cell is moderate, with Cohen's κ between 0.54 and 0.71 and overlap among solved challenges between 40% and 64%. Rates replicate; the identity of the solved challenges replicates less well, so we report rate-level rather than deterministic per-challenge capability claims.

These observations suggest that, at the evaluated resource ceiling, coordination overhead from repeated delegation, specialist restart, and evidence folding can offset potential benefits of maintaining a separate persistent planner. The controlled experiment therefore establishes the relative performance of the two architectures under this matched, cost-constrained operating point: adaptive escalation has a large positive effect in both architectures, whereas the planner-directed structural package does not provide an additional solve-rate advantage under the imposed cap. Because planner-directed runs reach that cap more frequently, relaxing the resource ceiling could change the relative solve rates and may allow additional planner-directed trajectories to complete; this counterfactual was not evaluated and should be tested directly in future work. The original full-split comparison remains an integrated-system result. The skill corpus is identical in all four conditions, so this experiment holds it constant and says nothing about its contribution either way.

6.4 Architectural trade-offs of a minimal persistent planner

The design occupies a middle position between a monolithic ReAct agent and a permanently staffed role-based team. A monolithic agent has the simplest communication structure but must keep planning, execution, and accumu-

lated observations in one context. A fixed multi-agent team can separate roles but creates persistent communication and memory boundaries that must be maintained throughout the run. The proposed design preserves only the planner and structured state. Expertise is materialized when needed as a fresh specialist plus a scoped skill.

This has two practical consequences visible in the traces. First, failed specialist trajectories can be discarded without discarding their conclusions; the evidence card is retained, while the verbose local context is not. Second, the planner's global view remains relatively stable even after thousands of commands. These are precisely the properties needed for long trajectories where raw shell output grows much faster than the useful decision state.

6.5 Skills, knowledge, and contamination

Procedural skills are attractive because they separate reusable security technique from model weights. They also complicate evaluation. CRAKEN shows that explicit external knowledge can materially improve CTF agents [13], while newer benchmarking work suggests that category tips can be neutral or harmful when a rich tool environment already supplies enough affordances [11]. Our behavioral traces show that specialists consult and act in category-consistent ways, but the present results cannot isolate how much skills improve success: the corpus is held constant across all four conditions of the controlled experiment, so it is part of the matched substrate rather than a manipulated factor.

Moreover, a public procedural corpus can overlap with public CTF solution knowledge. The leak audit detects environment shortcuts inside the run, not semantic overlap between skill notes, model pretraining, and challenge write-ups. Future evaluation should therefore combine skill ablations with fresh or live targets such as the evaluation philosophy proposed by CTFusion [10].

6.6 Execution surface is a first-class part of agent capability

The system is strongest on analysis-heavy tasks that fit the execute–inspect–revise loop and weaker when solving requires sustained interactive control. Binary exploitation has the lowest genuine category rate (12.8%) despite the highest cost per challenge record. Web also remains difficult when the target requires asynchronous or multi-actor interactions. EnIGMA's results show why this matters: supplying truly interactive security tools can unlock behavior unavailable through a simple shell-call interface [1]. The lesson is broader than this implementation: a penetration-testing agent should be evaluated as a coupled model–orchestrator–tool system, not as a model alone.

6.7 Raw versus genuine capability

A raw accepted flag is operationally interesting because it records that the agent found some path to the goal. But an environment leak is not evidence that the intended vulnerability was understood or exploited. The reduction from 41 to 29 is therefore not a cosmetic correction; it changes the main performance estimate by six percentage points. Public CTF evaluations that lack provenance auditing risk measuring benchmark fluency, environment opportunism, and exploitation skill as one undifferentiated number.

At the same time, the very behavior that invalidates a benchmark solve, that is enumerating unexpected files, inspecting containers, or reading exposed orchestration artifacts, resembles opportunistic behavior that may be valuable in a real authorized engagement. The appropriate response is not to hide the raw result, but to separate it from intended-path capability. We therefore report both and make the genuine result primary.

7 Threats to validity and limitations

Partial component isolation. The controlled experiment separates the planner-directed structural package from adaptive escalation under matched budgets. Decomposing that package into the planner, delegation, transient contexts and evidence folding would require a substantially larger factorial experiment. Crossing those four mechanisms with escalation gives 32 conditions; over the same 80-challenge subset and three passes, this would require 7,680 challenge-runs, compared with the 720 controlled challenge-runs reported here: 640 from two complete 2×2 passes and 80 from the additional standalone P1 robustness replication. Extending a three-pass 2×2 design to the full 200-challenge split would require 2,400 challenge-runs. The controlled comparison also fixes a resource operating point: the 340-action ceiling was chosen to bound per-challenge spend, and planner-directed runs reached that ceiling more frequently than the monolithic control. Consequently, the experiment does not establish whether the relative architecture effect would persist under a substantially larger action or spend budget.

Benchmark contamination. The manual audit removes identified environment leaks but cannot detect all pretraining exposure to public CSAW write-ups. CTFusion and related work show that this is a serious threat to static benchmark validity [10]. Fresh or live targets are needed for stronger claims.

Environment/tool confounding. Agent capability is coupled to installed tools, network topology, sandbox behavior, and the execution contract. Recent controlled work shows that richer security environments can change results substantially [11]. Our system's non-interactive one-command

contract likely suppresses performance on tasks requiring interactive I/O.

7.1 Interactive input/output

The one-command, non-interactive contract is the execution-surface limit that most constrains these results, visible in the Bandit levels that need a live terminal and in binary-exploitation challenges that must drive a process repeatedly. The restriction exists to make every action replayable, so lifting it is a question of preserving that property rather than of relaxing it: a structured session broker keeps an interactive episode under the same append-only record as a sequence of one-shot commands, with the harness rather than the agent owning the transcript and enforcing the timeouts, output limits and cleanup the current contract obtains by construction. The broker is implemented in the harness, described in Online Resource 1, and was not enabled for any run reported here, so the results characterise the non-interactive contract. The broker is expected to benefit most directly from challenge archetypes requiring sustained bidirectional interaction, including OverTheWire Bandit levels 24–25 and binary-exploitation tasks that require repeated stdin/stdout exchanges with a live process. Measuring what it recovers on the levels that currently fail on a live terminal is the immediate next step.

CTF-to-practice transfer. CTFs isolate exploit-oriented technical skills and provide unambiguous goals, while real penetration tests involve scoping, stealth constraints, incomplete documentation, multi-host identity relationships, reporting, and human communication. The present results support autonomous CTF solving as a proxy for security-testing competence, not unsupervised deployment on production networks.

8 Responsible-use considerations

Autonomous offensive-security research is dual use. The experimental design mitigates immediate risk by restricting agents to purpose-built CTF environments, verifying goals against benchmark ground truth, recording every action, and using persistent checkpoints that support later audit. These controls should be treated as minimum requirements rather than guarantees. Any transition from CTFs to organizational testing should require explicit authorization, network and credential scoping, reversible actions where possible, resource budgets, human escalation points, and retention of complete action logs.

The leak audit also suggests a governance lesson. An autonomous agent will exploit opportunities provided by its environment even when those opportunities are artifacts of the test harness. Production security-testing systems should therefore assume the agent will explore orchestration files, mounted volumes, service metadata, and other unintended surfaces unless policy and sandbox boundaries prevent it. Evaluation infrastructure should be designed as adversarially as the target environment itself.

9 Conclusion

We presented a planner-directed multi-agent LLM system for autonomous security testing in authorized CTF environments. The design maintains one persistent non-executing planner and creates skill-equipped specialists only on demand, with exactly one specialist active at a time. Persistent sandbox state, progressive skill disclosure, evidence folding, checkpointed memory, deterministic guardrails, and stall-triggered model escalation are integrated into the same execution loop.

The experimental suite shows that the integrated planner-directed system can sustain non-trivial autonomous offensive-security work with a small base model. On the 200-challenge NYU CTF Bench test split, it obtains 29 leak-audited genuine solves (14.5%) compared with 3 (1.5%) for the stateful GPT-4o-mini single-agent baseline. Because that comparison also varies model access and harness configuration, we ran a controlled architecture-by-escalation experiment under matched resources. Across two independent formal passes, the architectures remain level without escalation, while escalation improves solve rates in both and the monolithic control remains ahead with it enabled. This matched analysis shows a strong positive contribution from adaptive model escalation under the evaluated protocol, while the full-split comparison remains an integrated-system result. The execution record accompanying this result shows that specialist delegation, hint-consistent routing, adaptive escalation, evidence guards, and persistent state are exercised extensively during execution. The OverTheWire Bandit results provide a complementary long-horizon view: standalone performance exceeds sequential performance across the tested configurations, including 30/33 versus 24/33 for the planner-directed system and 27/33 versus 23/33 for the strongest single agent. Thus, heterogeneous task coverage and state-carrying continuity emerge as distinct dimensions of autonomous security-testing capability. Finally, provenance and cost materially affect how that capability should be interpreted: auditing reduces 41 raw accepted flags to 29 intended-path solves, while the $136.83 planner-directed run corresponds to $4.72 per genuine solve.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10207-026-01350-x.

Author Contributions A.A. contributed to the conceptualization, methodology, software implementation, experimental design and execution, data collection, analysis, and preparation of the initial manuscript draft. S.S.J. contributed to conceptualization, methodology, supervision, interpretation of results, and substantial review and revision of the manuscript. Both authors reviewed and approved the final manuscript.

Funding No funding was received for conducting this study.

Data availability The aggregate results and supporting analyses are provided in the manuscript and supplementary material. The underlying experimental records and execution traces are not publicly hosted because of their size, but may be made available by the corresponding author upon reasonable request, subject to practical storage, transfer, and security considerations.

Code availability The code is available in three tagged repositories: the evaluation harness, analysis, and released artefacts at https://github.com/iamamiramine/gencyber-Evaluation (`ijis-revision-v1.0`, commit `cdd5d97f0aa369e29c6ee9280f6e9eaaf3a63d11`); the agent at https://github.com/iamamiramine/gencyber-Agent (`gencyber-agent-v1.0`, commit `761b1fb7465d107736be783c126fe713a4a732aa`); and the sandbox and challenge API at https://github.com/iamamiramine/gencyber-Workbench (`gencyber-workbench-v1.0`, commit `5503caafa2afbeed0f1822cc7416e024402f58de`). The skill corpus is by Jagiełło [8], available under the MIT licence at https://github.com/ljagiello/ctf-skills, pinned at commit `1af14f9`. Online Resource 1 records the environment-pinning and dependency information.

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Ethical approval The study evaluates software agents exclusively in authorized CTF and wargame environments and does not involve human participants or animals.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Abramovich, T., Udeshi, M., Shao, M., Lieret, K., Xi, H., Milner, K., Jancheska, S., Yang, J., Jimenez, C.E., Khorrami, F., Krishnamurthy, P., Dolan-Gavitt, B., Shafique, M., Narasimhan, K.R., Karri, R., Press, O.: EnIGMA: interactive tools substantially assist LM agents in finding security vulnerabilities. In: Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., Zhu, J. (eds.) Proceedings of the 42nd International Conference on Machine Learning, Proceedings of Machine Learning Research, vol. 267, pp. 246–355. PMLR (2025)
2. Amine, A., Zayat, A., Saab, S.: Stateful llm agents for autonomous capture-the-flag solving. In: 2026 4th Cognitive Models and Artificial Intelligence Conference (AICCONF), pp. 1–8. IEEE (2026). https://doi.org/10.1109/AICCONF69182.2026.11600706
3. Anthropic: agent skills. Claude Platform Docs (2026). https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview. Accessed 8 August 2026
4. Bakker, I., Hastings, J.: Autonomous penetration testing: Solving capture-the-flag challenges with llms. In: 2025 Cyber Awareness and Research Symposium (CARS), pp. 1–6. IEEE (2025). https://doi.org/10.1109/CARS67163.2025.11337515
5. Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., Rass, S.: {PentestGPT}: evaluating and harnessing large language models for automated penetration testing. In: 33rd USENIX Security Symposium (USENIX Security 24), pp. 847–864 (2024)
6. Erdődi, L., Zennaro, F.M.: The agent web model: modeling web hacking for reinforcement learning. Int. J. Inf. Secur. **21**(2), 293–309 (2022). https://doi.org/10.1007/s10207-021-00554-7
7. Hilario, E., Azam, S., Sundaram, J., Imran Mohammed, K., Shanmugam, B.: Generative ai for pentesting: the good, the bad, the ugly. Int. J. Inf. Secur. **23**(3), 2075–2097 (2024). https://doi.org/10.1007/s10207-024-00835-x
8. Jagiełło, L.: ctf-skills: procedural skill packs for capture-the-flag technique. https://github.com/ljagiello/ctf-skills (2026). Commit 1af14f9, 1 May 2026. MIT licence
9. LangChain: deep agents overview. LangChain Documentation (2026). https://docs.langchain.com/oss/python/deepagents/overview. Accessed 8 August 2026
10. Lee, D., Bae, G.e., Yun, I.: CTFusion: a ctf-based benchmark for llm agent evaluation. In: Proceedings of the Second Workshop on Agents in the Wild: Safety, Security, and Beyond (AIWILD), ICML 2026 (2026)
11. Merves, T.H., Conaway, M.H., Escobar, J.M., Otal, H.T., Tatar, U.: Systematic capability benchmarking of frontier large language models for offensive cyber tasks. In: 2026 Systems and Information Engineering Design Symposium (SIEDS), pp. 1–6. IEEE (2026). https://doi.org/10.1109/SIEDS69358.2026.11540254
12. Shao, M., Jancheska, S., Udeshi, M., Dolan-Gavitt, B., Xi, H., Milner, K., Chen, B., Yin, M., Garg, S., Krishnamurthy, P.: Nyu ctf bench: a scalable open-source benchmark dataset for evaluating llms in offensive security. Adv. Neural. Inf. Process. Syst. **37**, 57472–57498 (2024). https://doi.org/10.52202/079017-1832
13. Shao, M., Xi, H., Rani, N., Udeshi, M., Putrevu, V.S.C., Milner, K., Dolan-Gavitt, B., Shukla, S.K., Krishnamurthy, P., Khorrami, F., et al.: Craken: cybersecurity llm agent with knowledge-based execution. arXiv preprint arXiv:2505.17107 (2025). https://doi.org/10.48550/arXiv.2505.17107
14. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in neural information processing systems, vol. 36, pp. 8634–8652. (2023). https://doi.org/10.52202/075280-0377
15. Sommervoll, Å.Å., Erdődi, L., Zennaro, F.M.: Simulating all archetypes of sql injection vulnerability exploitation using reinforcement learning agents. Int. J. Inf. Secur. **23**(1), 225–246 (2024). https://doi.org/10.1007/s10207-023-00738-3
16. Udeshi, M., Shao, M., Xi, H., Rani, N., Milner, K., Putrevu, V.S.C., Dolan-Gavitt, B., Shukla, S.K., Krishnamurthy, P., Khorrami, F., et al.: D-CIPHER: dynamic collaborative intelligent agents with planning and heterogeneous execution for enhanced reasoning in offensive security. arXiv preprint arXiv:2502.10931 (2025). https://doi.org/10.48550/arXiv.2502.10931

17. Wen, S.F., Shukla, A., Katt, B.: Artificial intelligence for system security assurance: a systematic literature review. Int. J. Inf. Secur. **24**(1), 43 (2025). https://doi.org/10.1007/s10207-024-00959-0
18. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (2023). https://openreview.net/forum?id=WE_vluYUL-X
19. Zhu, Y., Kellermann, A., Gupta, A., Li, P., Fang, R., Bindu, R., Kang, D.: Teams of llm agents can exploit zero-day vulnerabilities. In: Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 23–35 (2026). https://doi.org/10.18653/v1/2026.eacl-long.2

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.